

Exercícios de Revisão – Integração DevOps

Git, CI, Processos, Portas e Docker

Material de revisão para a disciplina de Integração DevOps

Objetivo. Revisar os principais conceitos trabalhados na disciplina: versionamento com Git/-GitHub, integração contínua com GitHub Actions, relação entre código e processos do sistema operacional, portas de rede e containerização com Docker. Este material **não inclui EC2 ou infraestrutura de nuvem.**

Parte A – Questões objetivas

1. Em um fluxo Git, qual comando cria uma cópia local de um repositório remoto?
 - A) `git pull`
 - B) `git clone`
 - C) `git commit`
 - D) `git status`
 - E) `git branch`
2. Qual comando mostra arquivos modificados, preparados para commit e ainda não rastreados?
 - A) `git log`
 - B) `git diff --cached`
 - C) `git status`
 - D) `git fetch`
 - E) `git remote`
3. O comando `git add arquivo.py` tem como função principal:
 - A) enviar o arquivo ao GitHub;
 - B) adicionar o arquivo à área de preparação (staging);
 - C) criar uma nova branch;
 - D) apagar o histórico anterior;
 - E) executar testes automatizados.
4. Em um repositório colaborativo, uma boa mensagem de commit deve:
 - A) ser genérica, como “alterações”;
 - B) explicar de forma breve e objetiva a mudança realizada;
 - C) conter sempre o nome do autor;
 - D) descrever apenas arquivos removidos;
 - E) substituir a documentação do projeto.
5. No GitHub Actions, um arquivo de workflow normalmente é armazenado em:
 - A) `/actions`
 - B) `/.github/workflows`
 - C) `/ci`
 - D) `/docker`

E) `/scripts`

6. Em integração contínua (CI), o objetivo principal é:

- A) executar manualmente todas as validações antes de cada commit;
- B) integrar alterações frequentes e validá-las automaticamente;
- C) impedir o uso de branches;
- D) substituir o Git;
- E) executar apenas o deploy em produção.

7. Um workflow configurado com `on: [push]` será executado quando:

- A) um container parar;
- B) ocorrer um push no repositório;
- C) o sistema operacional reiniciar;
- D) um pacote Python for instalado;
- E) uma porta de rede for fechada.

8. Em uma pipeline de CI para um projeto Python, qual sequência é mais coerente?

- A) checkout → configurar Python → instalar dependências → executar testes;
- B) testes → checkout → instalar Git;
- C) instalar Docker → apagar repositório → commit;
- D) configurar porta → criar branch → reboot;
- E) push → abrir navegador → editar código.

9. Ao executar `python -m http.server 8080`, o número 8080 representa:

- A) o PID do processo;
- B) a quantidade de arquivos servidos;
- C) a porta TCP utilizada pelo servidor;
- D) o número da versão do Python;
- E) o endereço IP da máquina.

10. Em Windows, o comando `netstat -ano | findstr :8080` é útil para:

- A) editar o código-fonte;
- B) identificar conexões e o PID associado à porta 8080;
- C) criar um container Docker;
- D) fazer commit automático;
- E) instalar dependências Python.

11. Se dois processos tentarem escutar a mesma porta TCP no mesmo endereço, normalmente:

- A) os dois funcionam sem conflito;
- B) o segundo tende a falhar com erro de endereço/porta em uso;
- C) o Git resolve o conflito;
- D) o Docker altera a porta automaticamente;
- E) o sistema cria outra rede.

12. Em relação a processo e programa, assinale a alternativa correta:

- A) programa e processo são exatamente a mesma coisa;
- B) programa é código armazenado; processo é uma instância em execução;
- C) processo existe apenas em containers;
- D) programa possui PID e processo não;
- E) um programa sempre gera exatamente um processo.

13. O arquivo `Dockerfile` é usado para:

- A) armazenar commits Git;
- B) descrever como construir uma imagem Docker;
- C) listar processos do Windows;
- D) executar apenas testes unitários;
- E) definir usuários do GitHub.

14. Qual comando constrói uma imagem Docker a partir de um `Dockerfile` no diretório atual?

- A) `docker run .`
- B) `docker build -t minha-app .`
- C) `docker ps -a`
- D) `docker stop .`
- E) `docker exec build`

15. A diferença essencial entre imagem e container é:

- A) imagem é executável e container não;
- B) imagem é um modelo imutável; container é uma instância criada a partir da imagem;
- C) container substitui o Git;
- D) imagem existe apenas na nuvem;
- E) não existe diferença prática.

16. No comando `docker run -p 8080:5000 minha-app`, a opção `-p 8080:5000` significa:

- A) copiar a porta 8080 para um arquivo;
- B) mapear a porta 8080 do host para a porta 5000 do container;
- C) iniciar dois containers;
- D) alterar o PID para 5000;
- E) abrir a porta 5000 no GitHub.

Parte B – Análise de cenários

- 17. Um aluno altera `app.py`, executa `git commit -m "corrige rota"` e recebe a mensagem de que não há alterações preparadas para commit. Qual etapa provavelmente foi esquecida? Explique o papel dessa etapa no fluxo do Git.
- 18. Um workflow de CI contém as etapas de checkout, configuração do Python e execução de `pytest`, mas os testes falham com erro de módulo não encontrado. Qual etapa provavelmente está ausente? Escreva um exemplo de comando para corrigi-la.
- 19. Uma aplicação web funciona quando executada diretamente com Python na máquina, mas dentro do container não pode ser acessada pelo navegador do host. Cite **duas causas possíveis** relacionadas a rede/porta e explique como verificá-las.

20. Um aluno executa `docker ps` e não encontra seu container, mas `docker ps -a` mostra que ele existe com status `Exited`. O que isso indica? Qual comando pode ajudar a investigar a causa?
21. Durante uma aula, um servidor local está usando a porta 8080. Ao tentar iniciar outro servidor na mesma porta, aparece um erro. Descreva um procedimento de diagnóstico no Windows para descobrir qual processo está ocupando a porta e como encerrá-lo de forma controlada.
22. Explique, em um fluxo DevOps, como Git, GitHub Actions e Docker podem trabalhar em conjunto desde a alteração de código até a validação automatizada da aplicação.

Parte C – Exercícios práticos

23. Crie um repositório Git local para um pequeno projeto Python. Registre pelo menos dois commits com mensagens claras e mostre o histórico com `git log -oneline`.
24. Crie uma branch chamada `feature-mensagem`, altere uma saída do programa, faça commit e depois integre a branch ao branch principal. Registre os comandos utilizados.
25. Escreva um workflow mínimo do GitHub Actions que, a cada `push`, faça checkout do código, configure Python 3, instale `pytest` e execute os testes.
26. Execute um servidor local com:

```
python -m http.server 8080
```

Em outro terminal, identifique a porta e o PID do processo. Depois finalize o processo e confirme que a porta foi liberada.

27. Para uma aplicação Python simples, escreva um Dockerfile contendo: imagem base Python, diretório de trabalho, cópia dos arquivos, instalação de dependências e comando de inicialização.
28. Construa uma imagem chamada `devops-revisao` e execute um container expondo a aplicação para o host. Registre os comandos `docker build`, `docker run`, `docker ps` e `docker logs` utilizados.